Image Forgery Detection (Deep Learning II Project)

Students Patrick Barry, Dinh Ha, Mahima Haridasan Sumathy
Supervisors Prof. Amaury Habrard, Erick Gomez

1 Introduction

Scientific trust comes down to honest data. Now that digital imaging dominates how we document experiments, manipulation has become a real problem. Forgery methods like splicing and cloning leave behind artifacts that are hard for people to spot but can completely distort a study's conclusions. That's why we need automated systems based on deep learning to detect these fakes and protect research integrity.

This project addresses the problem of image forgery detection by treating it as a binary semantic segmentation task. The objective is to classify every pixel of an input image as either "authentic" or "forged," thereby generating a precise mask that isolates manipulated regions. However, this task is complicated by the inherent nature of the data: forged regions often occupy a minuscule fraction of the total image area, leading to severe class imbalance, and the visual traces of manipulation are frequently high-frequency artifacts that blend seamlessly into complex biological textures.

To tackle these challenges, we investigate and compare three distinct encoder-decoder architectures: the classic U-Net, an optimised Efficient U-Net, and DeepLabV3+. We aim to balance model complexity with segmentation performance, testing lightweight solutions that can detect subtle forgeries while running on local hardware with a limited image dataset of size 5128.

The following chapters detail our journey from baseline implementation to architectural optimisation. We begin in Chapter 2 by formally defining the problem statement and the specific evaluation metrics required to assess performance on such imbalanced data. Subsequent chapters will cover our methodological approach, the ablation studies conducted to refine our loss functions and post-processing pipelines, and finally, a critical analysis of where our models succeed and where they fall short.

2 Problem Statement

The main technical challenge is automatically detecting forged regions in scientific images using binary semantic segmentation. Unlike standard classification tasks that output a single label for an entire image, our objective is to assign a precise class label, authentic (0) or forged (1), to every individual pixel within a given input. This treats forgery detection as a dense prediction task: the model generates a binary mask showing the exact boundaries of manipulated regions, like those from splicing or copy-move operations.

This segmentation task is complicated by two primary factors: visual subtlety and extreme class imbalance. Scientific forgeries often involve high-frequency artifacts or slight textural inconsistencies that blend seamlessly into the complex biological backgrounds of microscopy or macroscopic imagery. Furthermore, the manipulated regions typically occupy a negligible fraction of the total image area compared to the authentic background. Standard accuracy metrics are deceptive in this context; a model could achieve near-perfect accuracy simply by predicting the entire image as "authentic" (an all-zero mask), thereby failing to detect any forgery while statistically appearing successful.

2.1 Evaluation Metrics

To rigorously evaluate performance under these constraints, we rely on specialised metrics rather than simple pixel accuracy. The primary metric for this project is the Optimal F1 Score (oF1), as defined by the competition guidelines. Unlike standard semantic segmentation metrics, oF1 assesses the model's ability to separate distinct forged regions [1]. It utilises the Hungarian algorithm to optimally match predicted instances with ground-truth masks based on their overlap, penalizing both missed detections (false negatives) and phantom predictions (false positives).

To complement the instance-level oF1, we also consider standard pixel-wise metrics:

- **Dice Coefficient:** Measures the harmonic mean of precision and recall at the pixel level, providing a robust measure of boundary overlap.

$$Dice = \frac{2|P \cap G|}{|P| + |G|} \tag{1}$$

- **Intersection over Union (IoU):** Also known as the Jaccard Index, this metric provides a stricter penalty for segmentation errors than the Dice score.

$$IoU = \frac{|P \cap G|}{|P \cup G|} \tag{2}$$

- **Precision and Recall:** Reported to diagnose the specific nature of errors (e.g., whether the model is over-segmenting background noise or missing small forgery traces).

3 Methodology

3.1 Data Preprocessing

We standardised the dataset by generating zero-filled masks for authentic images and collapsing the 3D instance masks of forged images into single 2D binary maps using the maximum operation. To ensure consistent input dimensions, all samples were resized to 256×256 pixels; we utilized Bicubic interpolation for images to preserve textural details and Nearest Neighbor interpolation for masks to maintain binary integrity. Subsequently, pixel intensities were normalized to the $[0, 1]$ range to facilitate convergence, and the data was divided into training and validation sets using an 80/20 stratified split to ensure a balanced distribution of authentic and forged examples.

3.2 Model 1: U-Net

3.2.1 Model Architecture

We try U-Net [2] because it is one of the classic model developed for image segmentation task. U-Net is a Convolutional Neural Network (CNN) that was modified and extended to work with fewer training images and yield more precise segmentation. This model is well-suited for our project as our dataset is quite small: 2751 forged and 2377 authentic images, or 5128 in total. The structure of U-Net consists of a contracting path (encoder), a bottleneck, and an expanding path (decoder) with skip connections. Our U-Net implementation uses Pytorch and has the following structure:

The Contracting Path (Encoder) consists of four Downscaling stages to perform feature extraction from our input images. Our input layer is a Double Convolution (`DoubleConv`) block that takes 3 channels as input (RGB) and produces 64 feature maps. The `DoubleConv` block has the following structure: (i) a 3x3 `Conv2d` layer with kernel size 3 and padding 1 to keep the dimension of the feature maps consistent, followed by (ii) a `BatchNorm2d` and (iii) a `ReLU` activation to be able to capture non-linearity in the data. This sequence of operations is repeated twice, hence DoubleConv. Followed by the input layer are four `Down` blocks, each consisting of 2x2 `MaxPool2d` layer followed by a `DoubleConv` block. The number of channels doubles after each `Down` block ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$), while spatial resolution is halved. When using `MaxPool2d`, we remove redundant pixel information, so in order to compensate for the loss of spatial data, we increase the number of channels. Each channel corresponds to a filter that learns to identify a specific feature. By increasing the number of channels, the capacity of the U-net to understand more patterns increases.

The bottleneck is the deepest layer of our U-Net with 1024 channels. It is a transition point between the Contracting Path (encoder) and the Expanding Path (decoder). In our implementation, it is represented by the last `Down` and the first `Up` block. After four stages of Downsampling, the spatial resolution is at its lowest, the model now learns highly abstract features. The bottleneck contains the most complex semantic information about the image. It is here is that the model makes decision about whether an instance (area) of the image represents a forgery, based on the global context.

The Expanding Path (Decoder) restores spatial solution while adding localized details through skip connections. It consists of four `Up` blocks increasing spatial dimensions using Bilinear Interpolation (`Upsample`) or Transposed Convolution (`ConvTranspose2d`) with 2x2 kernel and stride 2. By default, we choose Bilinear for our U-Net because it does not require any parameters training. It calculate the value of a new pixel in the larger grid by a weighted average of the four nearest known pixels. This creates a smooth transition between pixel values, helpful for reconstructing the overall shape of a forged region. **Skip connections** make U-Net effective for image segmentation. Each of our `Up` block receive a feature maps from the corresponding `Down` block. The decoder pads the upsampled features to match the encoder's dimensions before performing the channel-wise concatenation. Following each concatenation, we use a `DoubleConv` to refine merged features. After the last `Up` block, an OutConv layer uses the `Conv2d` with kernel size 1x1 to map the final 64 channels feature map to our 2 classes (authentic and forgery). The output of our U-Net is a 4D Pytorch tensor with shape `(BatchSize, 2, H, W)`.

The illustration of the classic U-Net Architecture is provided in Figure 1.

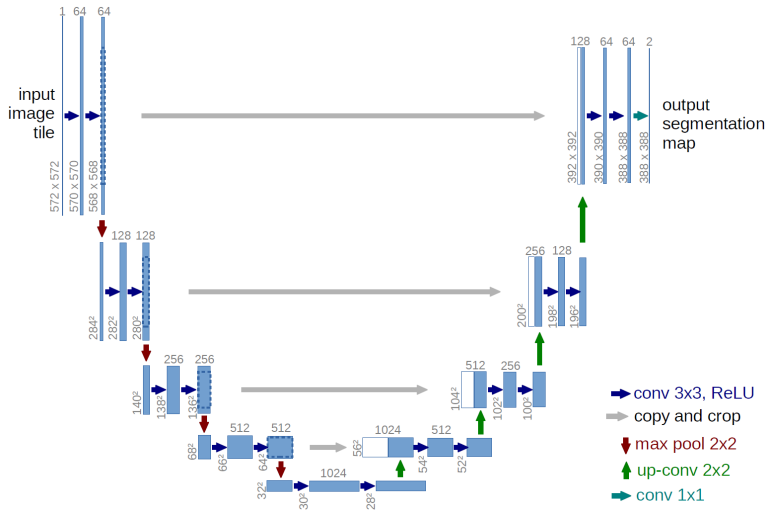


Figure 1: U-Net Architecture (in general, not our specific implementation)

3.2.2 Loss function

We had three options for the loss function: (i) **Weighted Binary Cross-Entropy (BCE)**, implemented as `BCEWithLogitsLoss` with an adaptive forgery weight [3] that dynamically adjusted based on the precision-recall ratio to regulate false positives; (ii) **Dice Loss**, defined simply as $1 - DiceScore$; and (iii) **BCE+Dice Loss**, a hybrid objective combining both terms.

3.2.3 Training

For the U-Net training, we train it on a Macbook Air M2 with 16 GB RAM. Instead of CUDA, we use its alternative on Mac which is `mps` (Metal Performance Shaders). We train our model using a batch of 16 images and for 10 epochs. Our training procedure is as follows: For each epoch, first, we perform one forward pass, the image goes through the network, which predicts a mask. Then we calculate the loss by choosing one of the three losses above. Then we backpropagate using the AdamW optimizer to adjust the model's weights in order to minimize the loss. We repeat this procedure for a number of epochs. To prevent overfitting, an **Early Stopping** based on the Dice score is added. If the score does not improve for a number of checks, the training stops and saves the best model (the model has the highest score). For better training, we have some **quick evaluation rounds** each epoch (using a subset of 25% the validation set) to update our learning rate (using a `ReduceLROnPlateau` scheduler) and forgery weight.

3.3 Model 2: EfficientU-Net

3.3.1 Model Architecture

Building on the U-Net baseline established in the previous section, we sought to optimise the architecture for computational efficiency without compromising segmentation performance. While we retained the core encoder-decoder structure and skip connections essential for localising scientific forgeries [2], our primary modification involved replacing the standard encoder with a more parameter-efficient alternative.

To achieve this, we implemented a U-Net with an **EfficientNet-B0** backbone pre-trained on ImageNet [4]. Our objective was to leverage the architecture's unique "compound scaling" strategy, which avoids the inefficiencies of arbitrarily scaling network dimensions. Unlike traditional models that might only increase depth (number of layers) or width (number of channels), EfficientNet uniformly scales depth, width, and resolution using a set of fixed scaling coefficients (α, β, γ) derived from a compound coefficient ϕ. By optimising these dimensions in a coordinated manner, the B0 variant achieves maximum representational power with minimal parameters, balancing the granular feature capture required for forgery detection with the memory constraints of our local hardware.

The core building block of this encoder is the Mobile Inverted Bottleneck Convolution (MBConv), which approaches standard residual learning differently. Instead of compressing representations inside the block, MBConv layers employ an "inverted bottleneck" design: they first expand low-dimensional inputs into a high-dimensional space using 1×1 convolutions to generate rich feature maps, and then project them back down after processing. Crucially, this processing is done via depthwise separable convolutions, a technique that decouples spatial filtering from channel mixing. This "depthwise trick" allows the model to learn complex

spatial hierarchies without the heavy computational cost of full convolution operations, reducing the parameter count to approximately 5.3M while maintaining deep network capabilities.

Each MBConv block integrates a Squeeze-and-Excitation (SE) optimisation module to enhance feature selectivity. This mechanism works by globally pooling spatial information ("squeezing") to create a summary of the feature map, which is then passed through a small bottleneck neural network to generate channel-wise attention weights ("excitation"). These weights are multiplied by the original feature map to selectively emphasise informative channels while suppressing irrelevant ones. In the context of scientific forgery, where manipulation traces can be extremely subtle, this self-attention mechanism is vital, enabling the network to focus specifically on the high-frequency artifacts often associated with splicing or cloning.

3.3.2 Training Procedure

To train our network effectively, we established a robust optimisation strategy tailored to the specific challenges of forgery detection, such as class imbalance and limited data. We utilised the AdamW optimiser, as it decouples weight decay from the gradient update, offering better generalisation than standard Adam [5]. We set a learning rate of $1e^{-4}$, which we found to be a stable starting point for fine-tuning the pre-trained EfficientNet encoder without destroying the learned features.

For our loss function, we adopted a compound loss strategy to address the fact that forged regions often occupy only a small fraction of the image. We combined Binary Cross-Entropy (BCE), which provides stable pixel-wise classification gradients, with Dice Loss, which directly optimises the intersection-over-union of the forged regions [6]. This combination ($L = 0.5 \cdot L_{BCE} + 0.5 \cdot L_{Dice}$) aims to balance pixel-level accuracy with the need to capture the overall shape of the manipulated areas.

Finally, to prevent overfitting on the relatively small dataset of 2,751 forged images, we implemented a data augmentation pipeline using the Albumentations library [7]. We applied augmentations (flips, shifts, rotations) exclusively to the training set to improve generalisation. The validation set was kept free of augmentations to ensure a stable and deterministic evaluation metric.

The exact specifications required to reproduce this model are summarised in Table 1.

Parameter	**Value/Setting**
Optimizer	AdamW
Learning Rate	1×10^{-4}
Batch Size	4
Loss Function	0.5 BCE + 0.5 Dice (Baseline)
Image Size	256×256
Augmentations	H-Flip ($p = 0.5$), V-Flip ($p = 0.5$), ShiftScaleRotate ($p = 0.5$)
Validation Metric	Optimal F1 (oF1) at Original Resolution

Table 1: Technical specifications for the experimental setup.

3.4 Model 3: DeepLabv3+

3.4.1 Model Architecture

We implement DeepLabV3+ an encoder-decoder structure with atrous convolutions that effectively captures multi-scale contextual information while maintaining spatial resolution [8].The architecture consists of three main components (i) Backbone Encoder - Feature extraction using pre-trained MobileNetV2 and Resnet50 (ii) Atrous Spatial Pyramid Pooling (ASPP) - Multi-scale context aggregation (iii) Decoder Module - Precise boundary refinement. We train our model with two different backbones MobileNetV2 and ResNet50.

Input Layer: The model accepts a three-channel RGB input tensor with dimensions 256×256 pixels, which undergoes ImageNet normalization using mean values of [0.485, 0.456, 0.406] and standard deviations of [0.229, 0.224, 0.225] per channel to standardize the input distribution for optimal pre-trained backbone compatibility.

Encoder (MobileNetV2 Backbone): Utilizing a pre-trained MobileNetV2 architecture [9], the encoder processes input through a sequence of seven inverted residual blocks with linear bottlenecks, progressively downsampling spatial resolution from 256×256 to 8×8 while expanding feature depth from 3 to 320 channels, with low-level features extracted at the 64×64 resolution level preserved for decoder integration.

(ResNet50 Backbone): Utilizing a pre-trained ResNet50 architecture [10], the encoder processes input through a hierarchy of four convolutional blocks with residual connections, progressively downsampling spatial resolution from 256×256 to 8×8 while expanding feature depth from 3 to 2048 channels. The architecture consists of an initial 7×7 convolution with stride 2 followed by max pooling, then four sequential blocks ($Conv2\_x$ through$Conv5_x$) containing 3, 4, 6, and 3 bottleneck residual units respectively, with low-level features extracted after the first max pooling operation at 64×64 resolution with 64 channels preserved for decoder

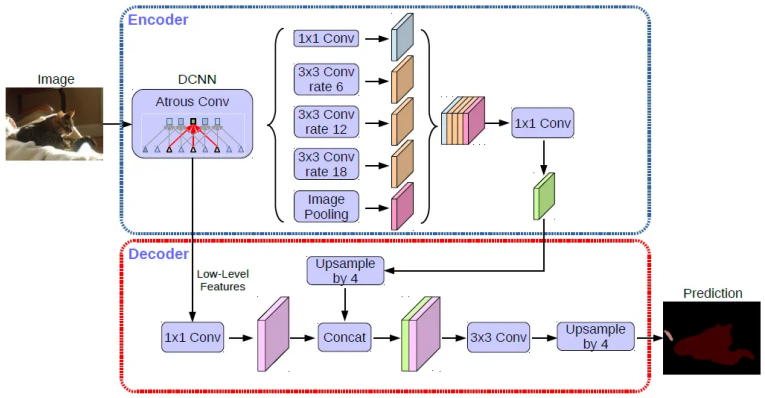


Figure 2: DeepLabv3+ Architecture

integration, and high-level features from the final $Conv2\_x$ block at 8×8 resolution with 2048 channels passed to the ASPP module for multi-scale context extraction.

Bottleneck (Atrous Spatial Pyramid Pooling - ASPP): The bottleneck module employs five parallel branches,four atrous convolutions with dilation rates of 6, 12, and 18 plus a global average pooling branch—that collectively capture multi-scale contextual information from the high-level encoder features, with all branches outputting 256 channels that are concatenated and compressed to form rich hierarchical representations.

Decoder: The decoder progressively restores spatial resolution through bilinear upsampling operations, first upsampling ASPP output 4× then concatenating it with 1×1-convolution-processed low-level features from the encoder, followed by two 3×3 convolutions for feature refinement and a final 4× upsampling to achieve the original input resolution of 256×256.

Output Layer: A final 1×1 convolution reduces the 256-channel decoder output to a single-channel prediction map, which then passes through a sigmoid activation function to generate pixel-wise probability values between 0 and 1, indicating the likelihood of each pixel belonging to forged regions for binary segmentation.

3.4.2 Training Procedure

Framework: We used PyTorch with Segmentation Models PyTorch (SMP) library. Encoder: Choosing ResNet50 with ImageNet pre-trained weights. ASPP Dilation Rates: [6, 12, 18] (default SMP configuration). Decoder Channels: 48 for low-level features, 256 for high-level features. Upsampling Method: Bilinear interpolation for all upsampling operations. Output Activation: Sigmoid for binary segmentation (forged vs authentic) Loss Function: We chose Binary Cross-Entropy (BCE) Loss. Batch Size: After facing crashes due to computational costs, we kept the batch size as 8. Training Epochs: We trained the model with 10 epochs to track the improvement of the learning process. Optimizer: Adam with learning rate 0.0001

Evaluation Metrics: We used Dice Coefficient and Intersection-over-Union (IoU) score to improve segmentation.

4 Experiments and Results

4.1 Experimental Setup

A critical aspect of our experimental setup was the handling of image resolution. Due to the memory constraints of our local training hardware, all models were trained on images resized to 256×256. However, scientific forgeries often involve subtle manipulations that can be lost at lower resolutions.

Therefore, during the evaluation phase, we upsampled our model's binary predictions back to the original image resolution before calculating the oF1 and Dice scores. This ensured that our reported metrics accurately reflect the model's performance on the full-scale scientific data, consistent with the competition's submission format.

All experiments were conducted using an 80/20 stratified split of the provided training data, ensuring a balanced distribution of authentic and forged examples in both sets.

4.2 Ablation Studies

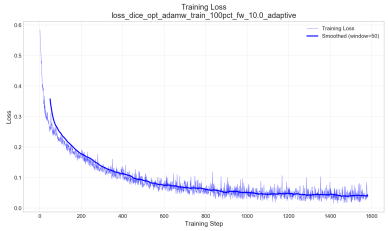
4.2.1 U-Net

We conduct a series of ablation studies to identify the optimal configuration for the classical U-Net architecture. U-Net is trained on a training set and a validate on validation set with a 80/20 split. It is important to note that the validation during training (which we call quick validation) is performed on a subset (25%) of the validation set.

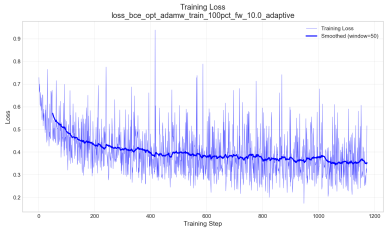
Loss Functions Analysis We experiment our U-Net training by using three different losses: Dice loss, BCE loss and Dice+BCE loss. The training losses for different loss functions are given in Figure 3 (top), while the validation results are given in Table 2. The validation results on Dice Score during training are given in Figure 3 (bottom). We oberseve these followings: **The Dice loss performs best** with roughly double the performance of BCE for oF1, Dice and IoU score. The Dice Score on quick validation also quickly increases up to 0.45 after only 100 training steps and plateaus. This makes sense because the Dice Formula is calculated based on the overlapping between the predicted and ground truth forgery pixels, therefore it handles well the class imbalance (most pixels are authentic with only small forged regions). Dice loss consistently decreases and plateaus at around 0.05. On the contrary, **the BCE loss struggles with class imbalance**, despite the fact that we already use a forgery weight and adaptive weighting to penalize the wrong forgery prediction. Its training loss curve shows high variance (around 0.3) and plateau around only 0.35. Its Dice score on quick validation also quickly plateaus at 0.2. Because of the poor performance of BCE loss alone, it is expected that **BCE+Dice loss is worse than Dice loss alone**. It peaks at the first quick validation then degrades afterward. It also triggered the early stopping much sooner (550 vs 1600 steps). This suggest the two losses have conflicting gradients and the model struggles to optimize both simultaneously.

Metric	Dice Loss	BCE Loss	Combined
oF1 Score	0.4468	0.2017	0.4023
Dice Score	0.4491	0.2257	0.3378
IoU Score	0.4428	0.1936	0.3320
Precision	0.0437	0.1150	0.0361
Recall	0.0116	0.1530	0.0176

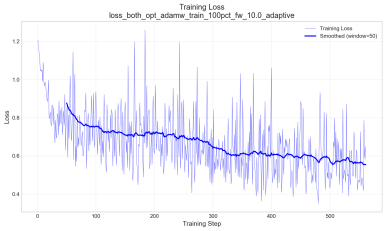
Table 2: U-Net ablation study on loss functions: Validation results on validation set after training



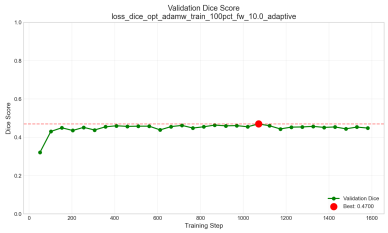
(a) Dice loss - Training loss



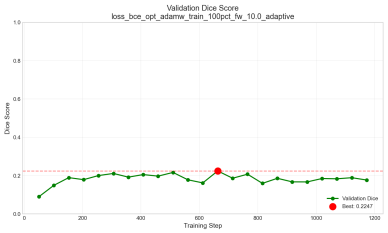
(b) BCE loss - Training loss



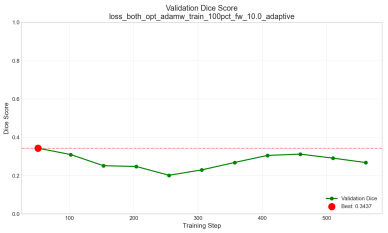
(c) Dice+BCE loss - Training loss



(d) Dice loss - Quick evaluation Dice score



(e) BCE loss - Quick evaluation Dice score



(f) Dice+BCE loss - Quick evaluation Dice score

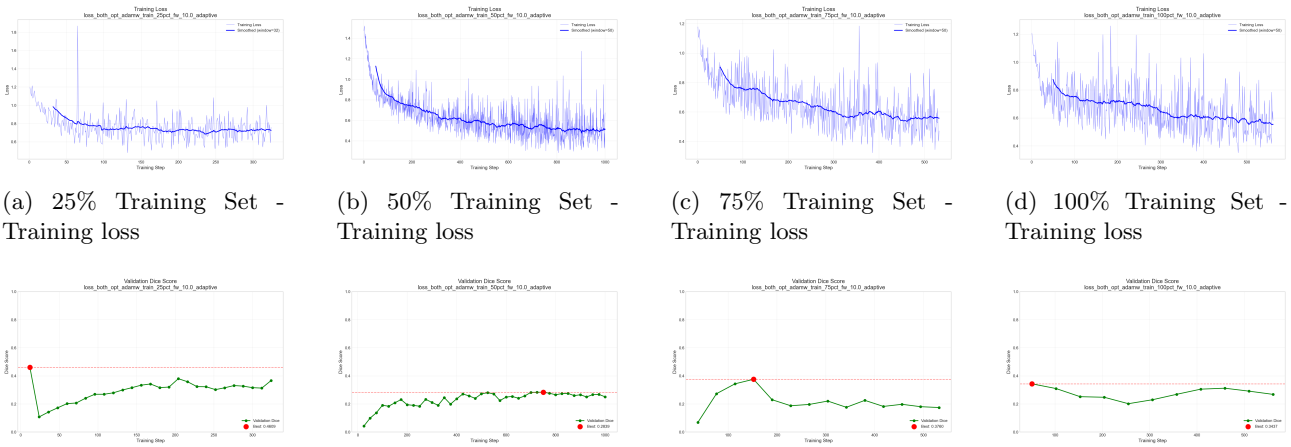
Figure 3: U-Net ablation study on loss functions: Training loss and quick validation on Dice scores (on 25% validation set)

Training Set Size Analysis We validate our U-Net with trained models with 25 %, 50 %, 75 % and 100 % of our training data. The training losses for these test sizes are given in Figure 4 (top), while the validation results are given in Table 3. The validation results on Dice Score during training are given in Figure 4 (bottom). We observed these following phenomenons: **The 50% training set shows distinctly different behavior**. It

performs worst across oF1, Dice and IoU, while better at precision and recall. It also trains much longer (1000 steps vs 320-560 for others), achieve its best Dice late (step 750), and shows the steadiest upward validation curved, yet its final metrics are the poorest. This suggests that it may converge to a local minimum. Our validation set is fixed, but for different quick validation, the model sees different examples. The 50% subset might happen to exclude training examples that are most similar to the validation set. **There is an early peaking pattern**. Three out of four models reach their peak very early, then degrade (25% at step 12, 75% at step 152, 100% at step 51). This indicates that our models overfit to training data quickly. **Precision and Recall are universally terrible** All runs show precision and recall in the 0.02-0.13 range. This means the model is either missing most forgeries (low recall), generating many false positives (low precision), or both. The oF1 and Dice being higher suggests the model gets some overlap right on the instances it does detect, but misses many entirely.

Metric	25% Train	50% Train	75% Train	100% Train
oF1 Score	0.3847	0.2685	0.3941	0.4023
Dice Score	0.3597	0.2743	0.3925	0.3378
IoU Score	0.3472	0.2468	0.3820	0.3320
Precision	0.0506	0.0958	0.0361	0.0361
Recall	0.0482	0.1315	0.0476	0.0176

Table 3: U-Net ablation study on training set sizes: Validation results on validation set after training



(a) 25% Training Set - Training loss

(b) 50% Training Set - Training loss

(c) 75% Training Set - Training loss

(d) 100% Training Set - Training loss

(e) 25% Training Set - Quick evaluation Dice score

(f) 50% Training Set - Quick evaluation Dice score

(g) 75% Training Set - Quick evaluation Dice score

(h) 100% Training Set - Quick evaluation Dice score

Figure 4: U-Net ablation studies: Training loss and quick validation on Dice scores (on 25% validation set) for different training test sizes

4.2.2 EfficientU-Net

We conducted a series of ablation studies to identify the optimal configuration for the EfficientU-Net architecture. All results reported below are based on the Optimal F1 (oF1) score evaluated at the original image resolution.

Loss Function Analysis Contrary to common practice in segmentation tasks where Dice Loss is preferred for imbalanced data, our experiments for this model revealed that Binary Cross-Entropy (BCE) yielded the highest performance on this specific dataset. As illustrated in Figure 5 (left), the model trained with pure BCE achieved a validation oF1 of 0.479, significantly outperforming the Dice-only model (0.398) and the Combined loss (0.425).

We hypothesise that this performance gap is driven by the dataset's high proportion of authentic images ($\approx 46\%$). Dice loss formulations can become unstable or provide weak gradients when the ground truth mask is empty (authentic), as the loss has an intrinsic bias toward specific extremely imbalanced solutions. Furthermore, literature suggests that Dice often has difficulty adapting to different distributions of region sizes, whereas Binary Cross-Entropy (BCE) implicitly encourages the ground-truth region proportions [11]. In our case, BCE provides a robust pixel-wise supervision signal for authentic images, effectively suppressing false positives and allowing

the model to achieve near-perfect scores on the non-forged subset by accurately reflecting the "zero-forgery" distribution.

Impact of Pre-training To quantify the benefit of transfer learning, we compared our ImageNet pre-trained EfficientNet-B0 backbone against a model trained from scratch. The pre-trained model demonstrated superior convergence and generalisation (Figure 5, center), achieving a best oF1 of 0.479 compared to 0.464 for the scratch model. This confirms that the low-level features (edges, textures) learned from natural images are transferrable to the domain of scientific forgery detection.

Post-Processing and Scaling Further analysis of the model's predictions revealed a tendency to output low-confidence "noise" artifacts in authentic regions. We addressed this through two post-processing optimisations, detailed in Figure 6:

1. **Thresholding:** A grid search indicated that a stricter binarisation threshold of 0.7 was superior to the standard 0.5 (Figure 6, left). This improved the oF1 to 0.481 by effectively filtering out low-confidence background noise which the BCE loss had not fully suppressed.
2. **Minimum Region Size:** By filtering out predicted instances smaller than 500 pixels, we further improved the score to 0.484. This monotonic improvement (Figure 6, right) confirms that the model occasionally predicts tiny, scattered false positives which degrade the instance-level metric.

Finally, we observed a linear relationship between training set size and performance (Figure 5, right), with no sign of saturation at 100% data usage. This suggests that the EfficientU-Net capacity is not yet maximised and would likely benefit from a larger dataset.

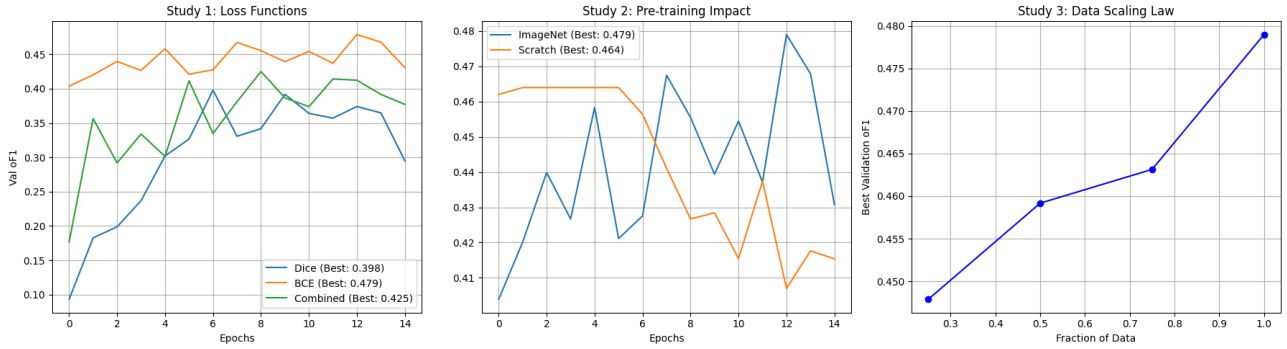


Figure 5: Ablation studies on training configuration. **Left:** BCE loss outperforms Dice and Combined loss. **Center:** ImageNet pre-training leads to faster convergence. **Right:** Performance scales linearly with data volume.

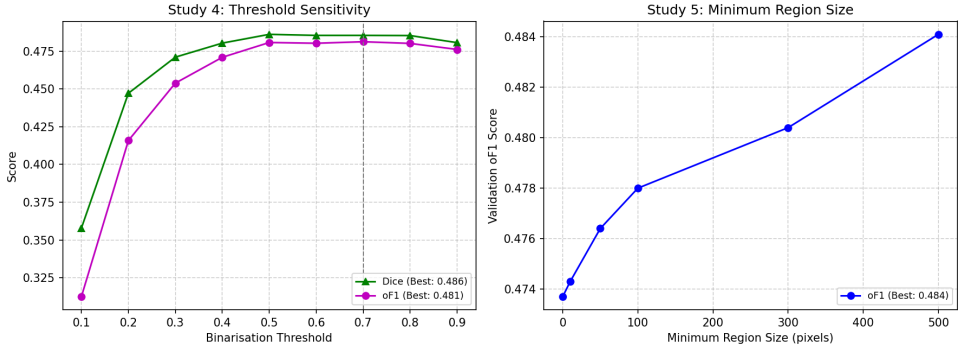


Figure 6: Analysis of post-processing techniques. **Left:** Sensitivity of oF1 to binarisation threshold, peaking at 0.7. **Right:** Impact of minimum region size filtering, showing consistent gains by removing small noise artifacts (< 500 pixels).

Qualitative Error Analysis To understand the model's limitations, we visually inspected failure cases (see Appendix B, Figure 10). While the model demonstrates high precision (rarely predicting false positives on authentic images), it occasionally exhibits False Negative behavior.

In the top row (microscopy sample), the model fails to detect the manipulated regions entirely, likely due to the high textural similarity between the forged area and the background. Similarly, in the bottom rows (macroscopic samples), the model successfully identifies the most prominent forgeries but misses secondary, subtler manipulations. This behaviour is consistent with our quantitative findings: the use of BCE loss and a strict 0.7 threshold optimises the model to aggressively suppress noise, which is crucial for the authentic dataset subset, but this comes at the cost of occasionally missing lower-confidence forgery instances.

Boundary Precision and Instance Separation Qualitative analysis of the failure cases (Appendix B, Figure 10) provides critical insight into the model's segmentation characteristics. Regarding instance separation, the model performs well; as seen in the bottom row of Figure 10, multiple predicted regions remain spatially distinct with no evidence of incorrect merging, confirming the effectiveness of the connected component labelling. However, in terms of boundary precision, the model exhibits a tendency towards under-segmentation. A close comparison of the macroscopic samples (middle and bottom rows) reveals that the predicted masks are consistently thinner and smaller than the ground truth. This boundary erosion is a direct consequence of the strict binarisation threshold (0.7): while it may eliminate background noise, it also prematurely cuts off the edges of valid forged regions where pixel confidence naturally tapers off.

4.2.3 DeepLabV3+

In DeepLabV3+ model we took two differenct backbone architectures (MobileNetV2 and Resnet50) and compared their results. Figure 7 and Figure 8 shows the Loss convergences and segmentation metrices for both the architectures.

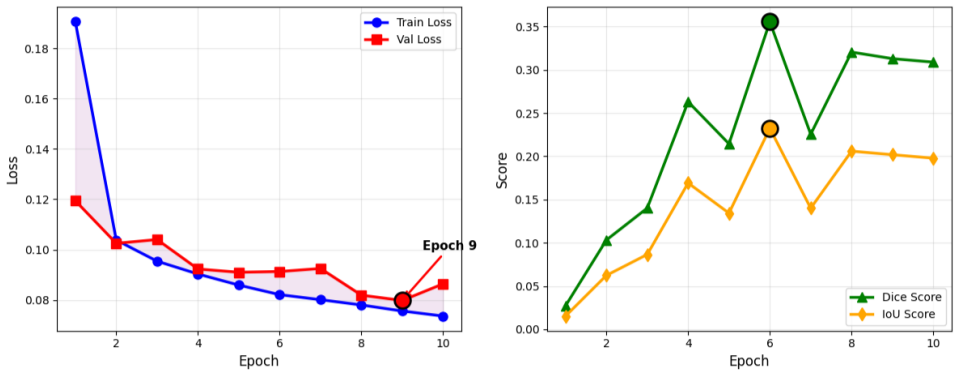


Figure 7: MobileNetV2 Loss Convergence and Segmentation Metrices

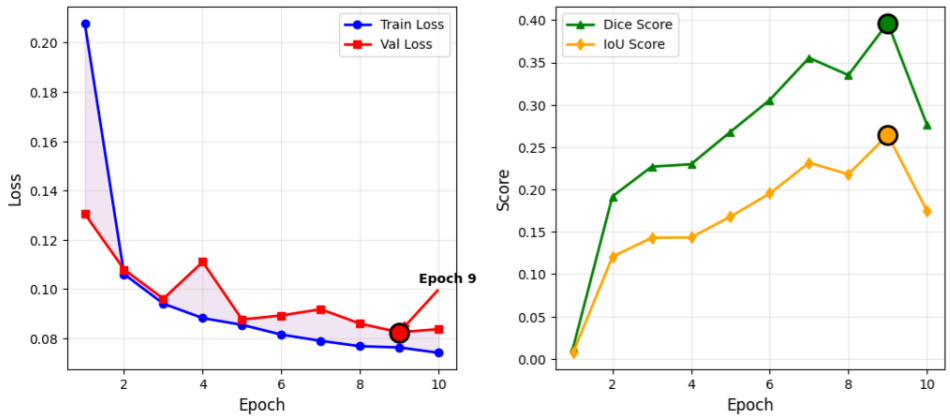


Figure 8: Resnet50 Loss Convergence and Segmentation Metrices

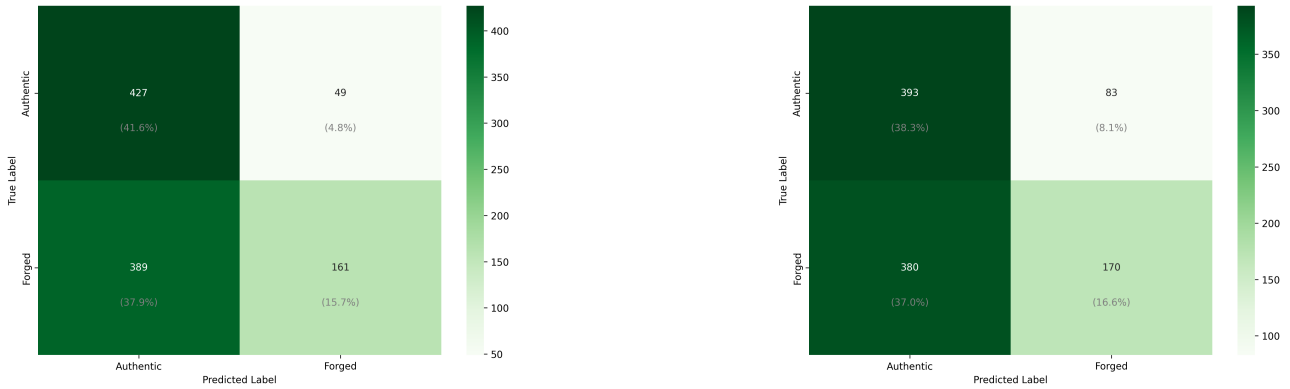
ResNet50 has 26 million parameters, while MobileNetV2 has about 6 million, the loss convergence shows overfitting in MobilNetV2 characterized by its early peak at Epoch 6 and subsequent instability, whereas ResNet50 demonstrates a steadier learning curve and a higher peak Dice score of 0.3964.

A Dice score of 0.39 is still relatively low this is due to class imbalance.The authentic images overwhlems forged images.In future to improve the performance advanced data augmentation techniques can be used and currently the BCE loss function treats every pixel the same, so further enhancement of loss function like Dice loss+ Focal loss optimises the overlaps.

Our comparative evaluation Tabel4 shows that DeepLabV3+ with ResNet50 backbone achieves superior performance over MobileNetV2, with an oF1 score of 0.524 compared to 0.499 (4.97% absolute improvement). The critical challenge lies in recall (33-34%), indicating that approximately two-thirds of forged regions remain undetected. Figure 9 shows the Instance level confusion matrix of both models we can see the models does similar performance in classifying authentic and forged images. These results establish a baseline for image forgery detection and highlight the need for advanced techniques to improve recall while maintaining precision.

Table 4: Performance and Segmentation Metrics Comparison of Models

Model Backbone	**OF1 Score**	**Precision**	**Recall**	**Dice Score**	**IoU Score**
MobileNetV2	0.498886	0.616686	0.329154	0.370238	0.245444
ResNet50	0.523694	0.623454	0.343433	0.367714	0.242641



(a) MobileNetV2 Model

(b) ResNet50 Model

Figure 9: Instance level Confusion Matrix

5 Conclusion

This project explored the efficacy of deep learning architectures for the automated detection of scientific image forgeries. We progressed from a baseline U-Net implementation to more sophisticated architectures, including DeepLabV3+ and an optimised Efficient U-Net. Our experiments conclusively demonstrated that architectural complexity does not guarantee superior performance on small, imbalanced datasets. The Efficient U-Net, utilising an EfficientNet-B0 backbone, emerged as the most effective solution, achieving a validation oF1 score of 0.484. It significantly outperformed both the standard U-Net and the heavier ResNet50-backed DeepLabV3+.

Our best models each had different strengths. The Efficient U-Net struck a good balance between speed and stability. The DeepLabV3+ with ResNet50, on the other hand, was the best at actually finding forgeries, achieving the highest oF1 score (0.524) thanks to its deeper architecture and better feature extraction. But all our models shared the same major problem: low recall. The result was masks that didn't fully cover the forged areas.

Despite these successes, our solution remains a baseline with clear limitations. Qualitative analysis revealed that while the model is highly precise, it occasionally fails to detect subtler manipulations in high-texture microscopy samples, exhibiting a trade-off where the high thresholds used to suppress noise also suppress faint forgery traces. Future work to address these shortcomings would prioritise the implementation of advanced data augmentation techniques to improve generalisation and the exploration of hybrid loss functions, such as Focal Loss combined with Dice, to better handle the hard-example mining. Additionally, given more computational resources and time, we would extend our research to include larger, state-of-the-art architectures such as Vision Transformers (e.g., SegFormer) or Mask2Former, which may offer superior capability in capturing the global context necessary for identifying complex contextual anomalies.

References

[1] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955.

[2] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015.

[3] Salman H. Khan, Munawar Hayat, Mohammed Bennamoun, Ferdous Sohel, and Roberto Togneri. Cost sensitive learning of deep feature representations from imbalanced data, 2017.

[4] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks, 2020.

[5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.

[6] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. V-net: Fully convolutional neural networks for volumetric medical image segmentation. In *2016 Fourth International Conference on 3D Vision (3DV)*, pages 565–571, 2016.

[7] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, February 2020.

[8] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation, 2018.

[9] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks, 2018.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

[11] Bingyuan Liu, Jose Dolz, Adrian Galdran, Riadh Kobbi, and Ismail Ben Ayed. Do we really need dice? the hidden region-size biases of segmentation losses. *Medical Image Analysis*, 91:103015, 2024.

[12] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. *European Conference on Computer Vision (ECCV)*, pages 801–818, 2018.

A Contribution and Planning

A.1 Contributions

All group members contributed equally to the project's research, development, and analysis phases. The specific model architectures implemented by each member, along with contribution percentages, are listed below:

- **Dinh Ha (33.3%):** Standard U-Net implementation.
- **Patrick Barry (33.3%):** EfficientU-Net (U-Net with EfficientNet-B0 backbone).
- **Mahima Haridasan Sumathy (33.3%):** DeepLabV3+ (MobileNetV2 and Resnet50 backbone).

A.1.1 Initial Plan

The project was originally structured into three distinct sprints to ensure a logical progression from baseline implementation to optimisation.

- **Phase 1 (Foundation):** Setup of the development environment, implementation of the U-Net baseline, and establishment of validation protocols.
- **Phase 2 (Optimization):** Execution of scaling experiments, integration of the Hungarian Matching algorithm for oF1 calculation, and conducting ablation studies on augmentations and loss functions.
- **Phase 3 (Analysis):** Final performance evaluation, qualitative error analysis, and report synthesis.

A.1.2 Execution of Plan

While the overall division of tasks (Foundation, Optimization, Analysis) remained consistent with our initial proposal, the timeline underwent a necessary adjustment. Due to concurrent academic deadlines in mid-January, the execution window for this project was compressed into a two-week sprint following the submission of the MLDM project.

To accommodate this compressed schedule, we adopted a more focused experimental strategy. Instead of implementing a wide range of models, we prioritised a small subset of models based on early literature review. This approach allowed us to complete the training and ablation phases efficiently, preserving a final buffer period which we dedicated to metric verification and report writing.

B Additional Visualisations

C Check list

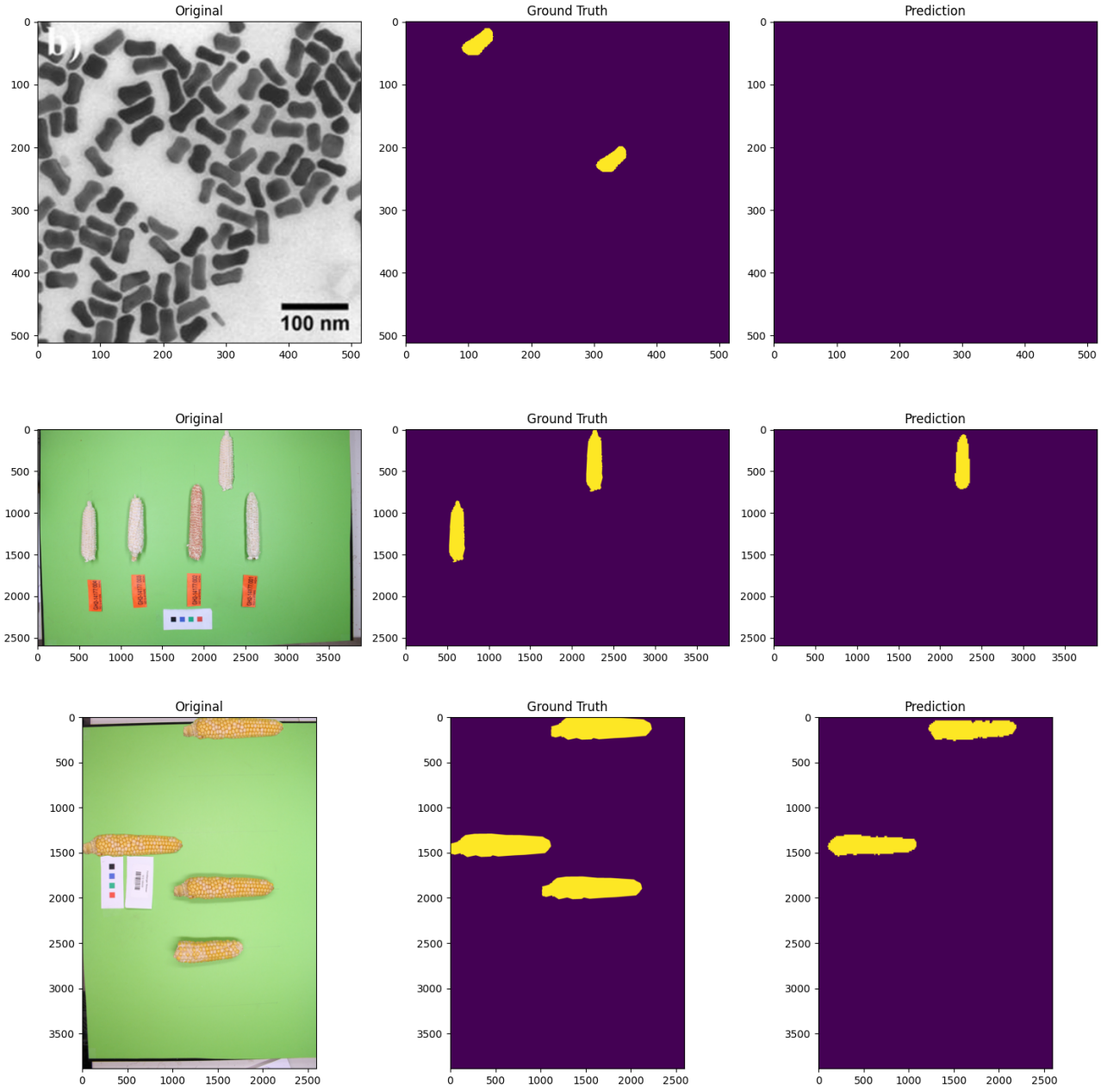


Figure 10: Qualitative visualisation of failure cases of EfficientU-Net. **Top Row:** A complete false negative where the model misses both spliced regions in a TEM image. **Middle & Bottom Rows:** Partial detections where the model identifies the primary forgery but misses additional instances. These examples highlight the precision-recall trade-off inherent in the high-threshold configuration.

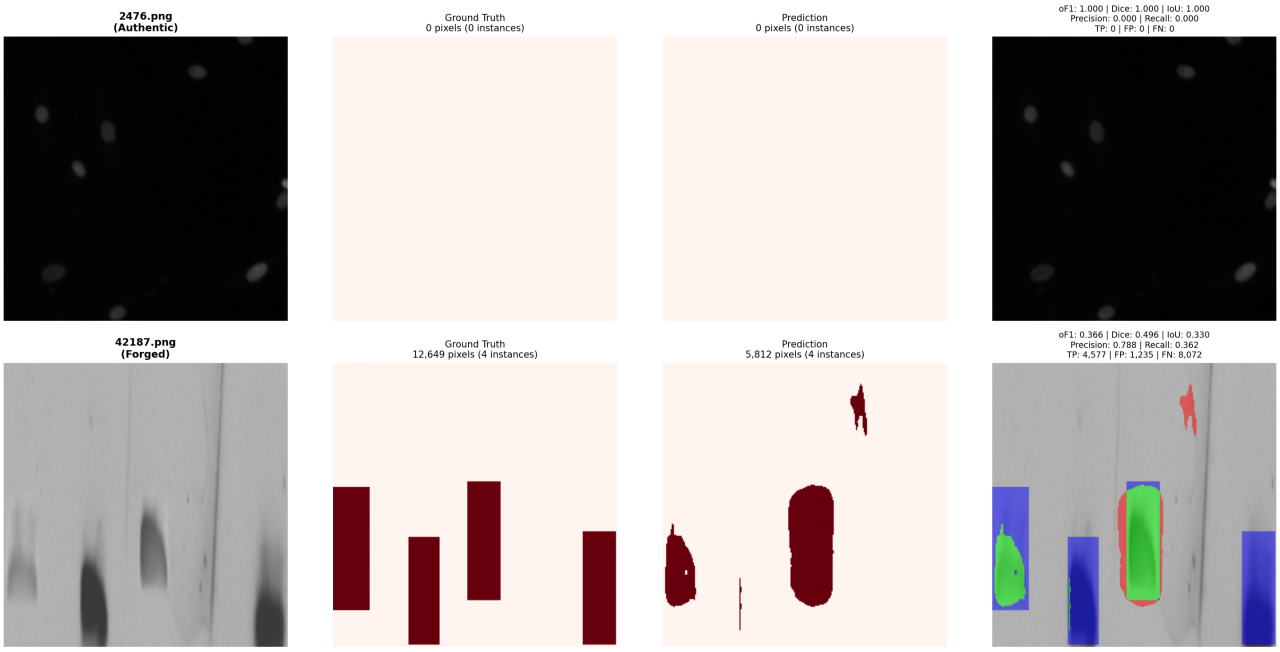


Figure 11: Qualitative visualisation of failure cases of DeepLabV3+(Resnet50)

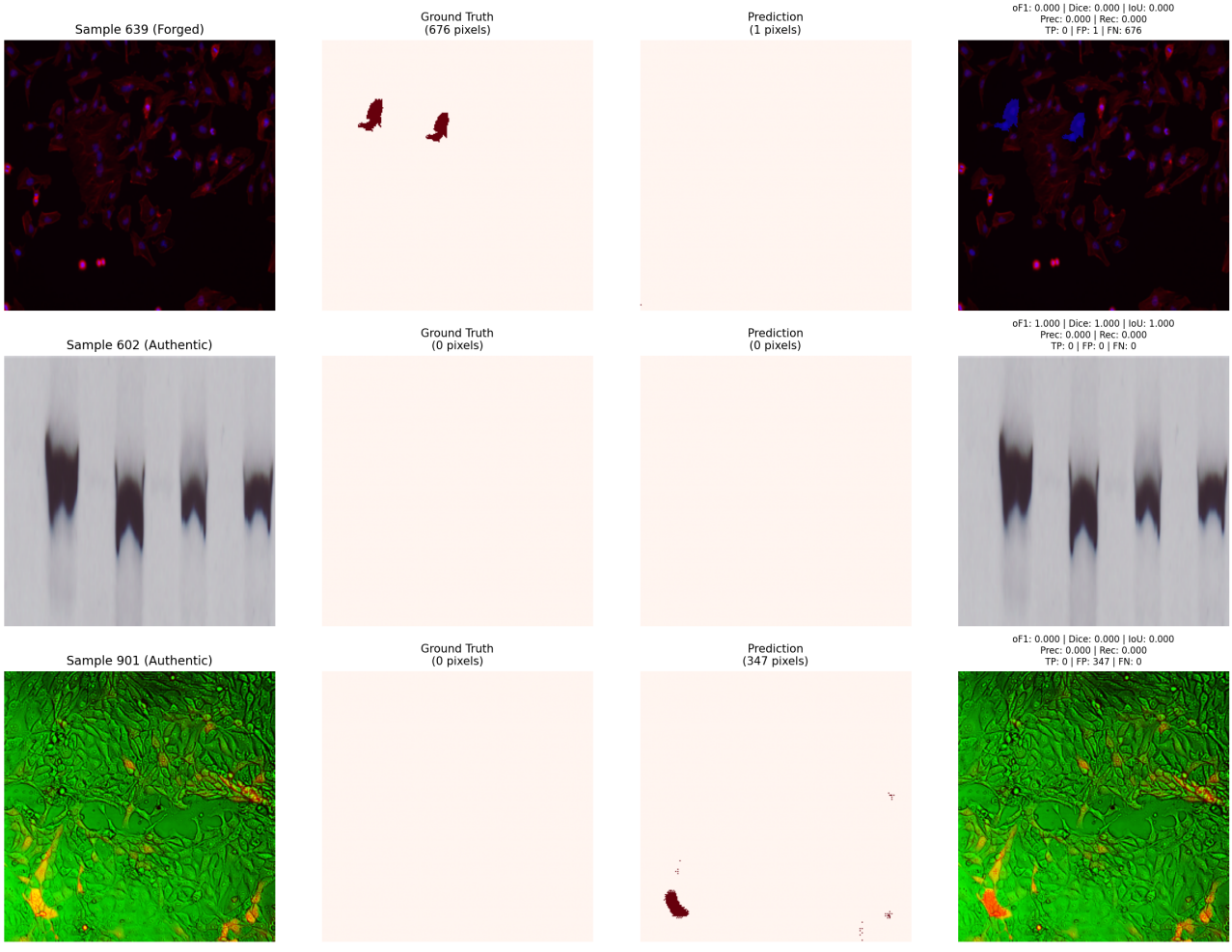


Figure 12: Qualitative visualisation of failure cases of U-Net

✓ or ✗	**Requirement**	**Comments/Observations**
✓	Did you proofread your report?	Yes.
✓	Did you present the global objective of your work?	The overall objective was outlined.
✓	Did you present the principles of all the methods/algorithms used in your project?	Each algorithm has been described in detail.
✓	Did you cite correctly the references to the methods/algorithms that are not from your own?	Yes.
✓	Did you include all the details of your experimental setup to reproduce the experimental results, and explain the choice of the parameters taken?	Each choice was made explicit and detailed sufficiently.
✓	Did you provide curves, numerical results and error bars when results are run multiple times?	Yes, all results have been provided.
✓	Did you comment and interpret the different results presented?	All results were interpreted and discussed.
✓	Did you include all the data, code, installation and running instructions needed to reproduce the results?	The code has been uploaded to Moodle along with the report.
✓	Did you engineer the code of all the programs in a unified way to facilitate the addition of new methods/techniques and debugging?	To the best of our abilities.
✓	Did you make sure that the results of different experiments and programs are comparable?	Yes, we ensured all results came from the same controlled settings.
✓	Did you comment sufficiently your code?	Yes.
✓	Did you add a thorough documentation on the code provided?	Yes, a readme file was provided with the code.
✓	Did you provide the provisional planning and the final planning in the report and discuss organization aspects in your work?	These were added and discussed.
✓	Did you describe precisely in the report the algorithms that were not seen in class?	The algorithms used have been detailed precisely.
✓	Did you provide the workload percentage between the members of the group in the report?	Yes.
✓	Did you send the work in time?	Yes.